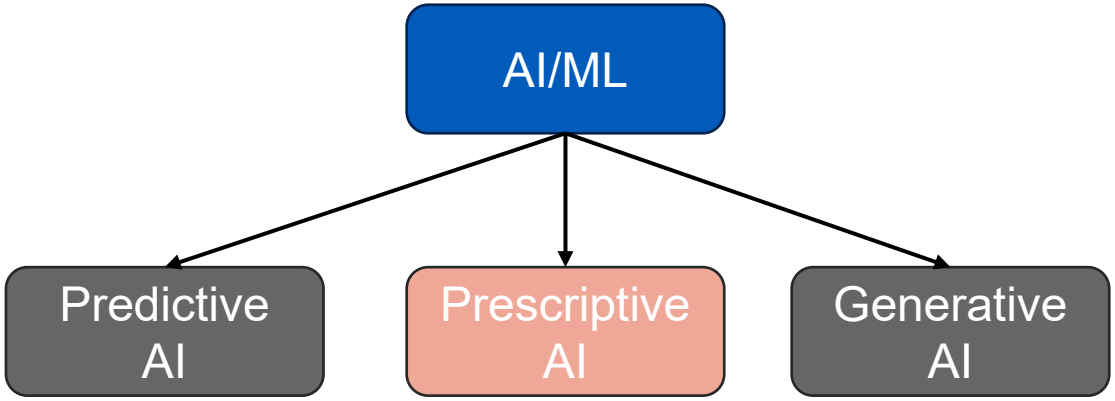
CE 400/500

Special Topics: AI for Chemical Engineers

Lecture 16: Introduction to Prescriptive AI, Genetic Algorithm, Bayesian Optimization

University at Buffalo
The State University of New York

Prescriptive AI: From Prediction to Optimization



Prescriptive AI is the next step beyond prediction. **It recommends the best action to take to achieve a goal.**

It is widely used in reactor (synthesis) optimization, material performance optimization and discovery, control systems, and decision-making under uncertainty. It is the **foundational** technology to enable the future fully autonomous chemistry/material laboratories.

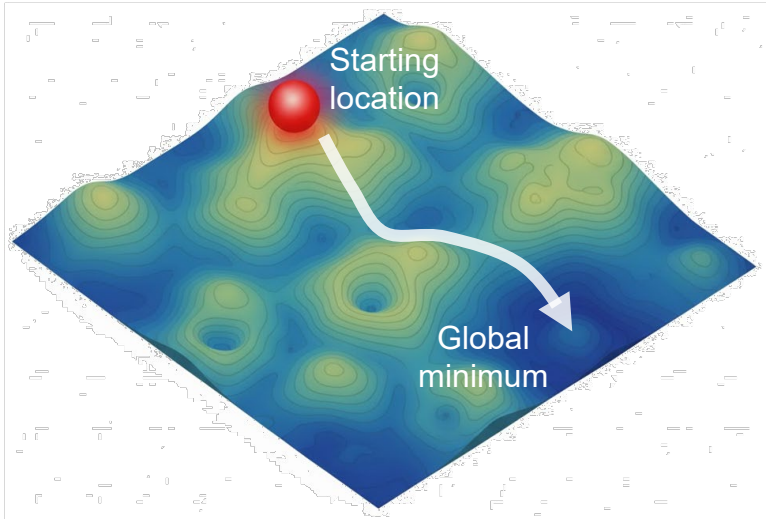
https://developers.google.com/machine-learning/intro-to-ml/what-is-ml

What Is Optimization?

Optimization seeks the **best input** $\mathbf{x}^*$ that minimizes or maximizes an objective $f(\mathbf{x})$:

$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}}\, f(\mathbf{x}) \quad \text{or} \quad \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}}\, f(\mathbf{x})$$

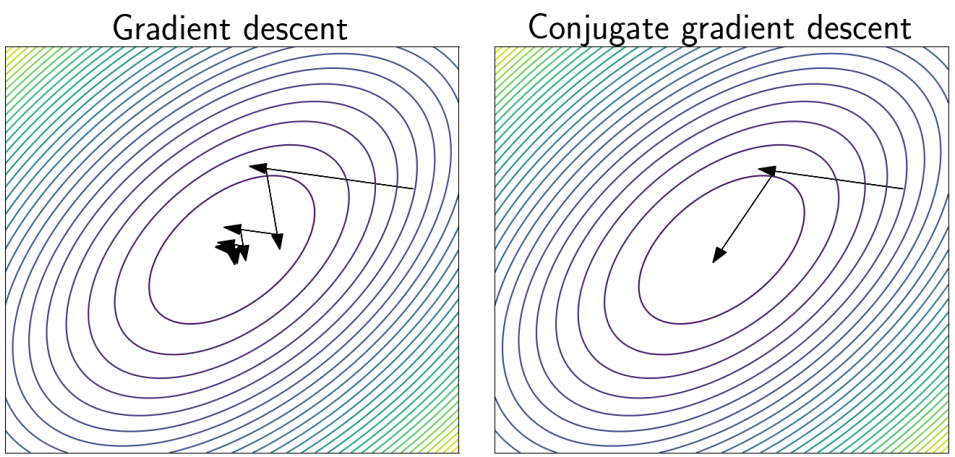
- x: input feature vector/design variables (e.g., temperature, chemical composition)
- $f(\mathbf{x})$: objective or "fitness" function
  - $f(\mathbf{x})$ could be a learned predictive model (e.g., MLP model for predicting H_2 storage in MOFs)
  - Prescriptive AI does not require explicit knowledge of $f(\mathbf{x})$; instead, it explores and learns by interacting with the system or simulation to discover optimal solutions.
- May be constrained by physical, safety, or resource limits



Categories of Optimization Algorithms

Optimization approaches can be broadly grouped as:

- **Deterministic / Gradient-based:** Requires analytical derivatives (e.g., gradient descent, conjugate gradient). Efficient but limited to smooth, differentiable problems.



Credit: Andy Jones

Gradient descent (GD):

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \nabla f\left(\mathbf{x}^{(t)}\right)$$

If feature vector for sample i is $\mathbf{x_i} = \{x_{i,1}, x_{i,2}, \ldots\}$, GD becomes:

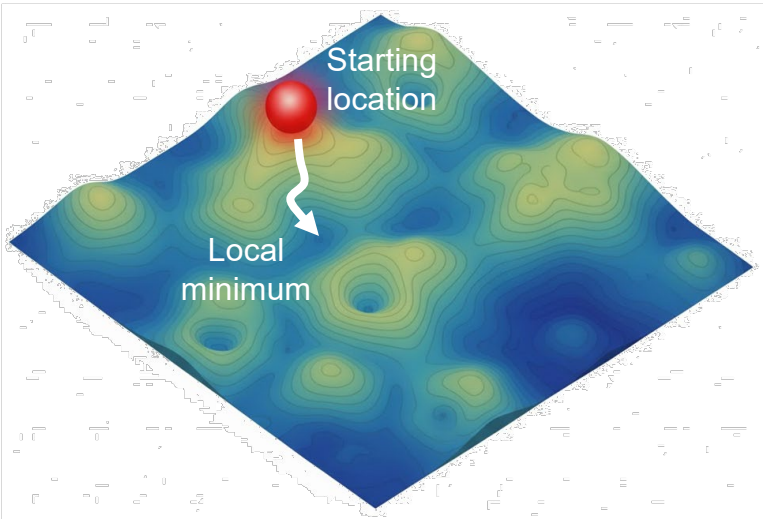
$$x_{i,1}^{(t+1)} = x_{i,1}^{(t)} - \alpha \frac{\partial f}{\partial x_{i,1}^{(t)}}$$

Check out this demo about GD! (made with Claude Artifacts)

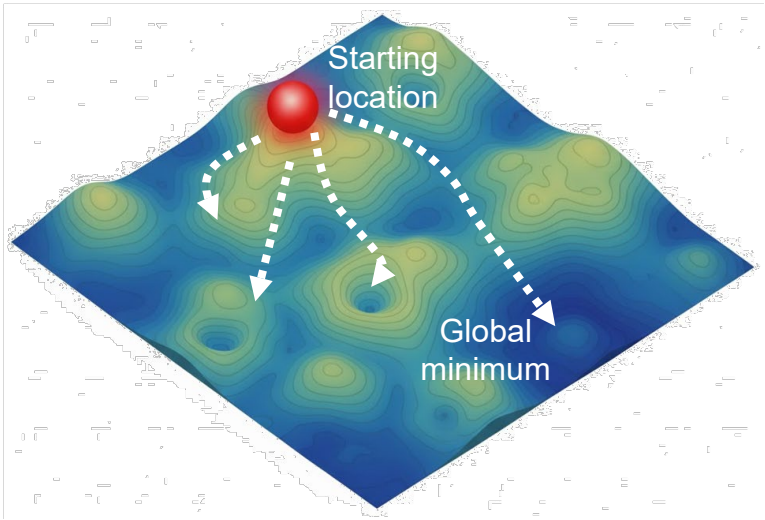
Categories of Optimization Algorithms (cont'd)

- **Stochastic / Derivative-free:** Works for black-box or noisy systems. Common methods includes genetic algorithm (GA) and Bayesian optimization (BO).

Prescriptive AI mainly leverages stochastic algorithms because real-world engineering systems are nonlinear, discontinuous, and "black-box".

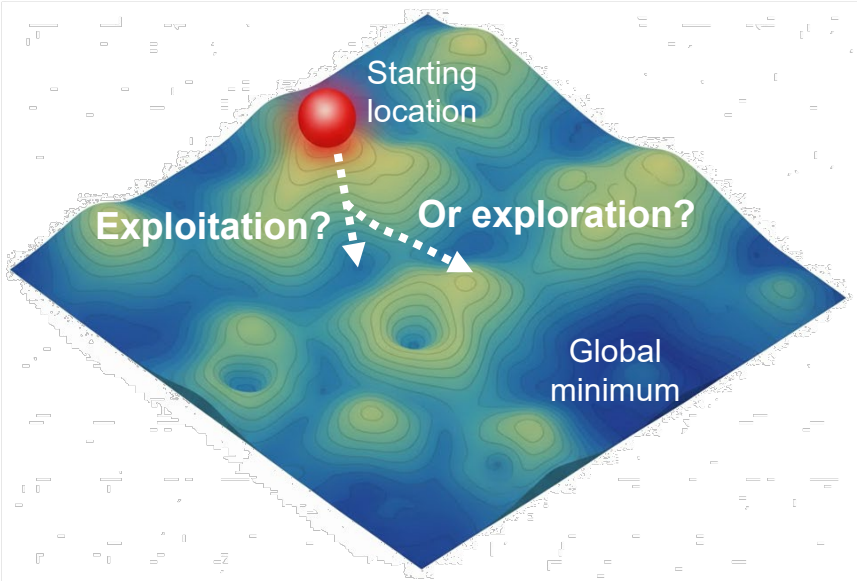


Deterministic
(e.g., gradient descent)



Stochastic
(e.g., genetic algorithm)

Exploration-Exploitation Tradeoff in Stochastic Optimization



All stochastic algorithms must balance:

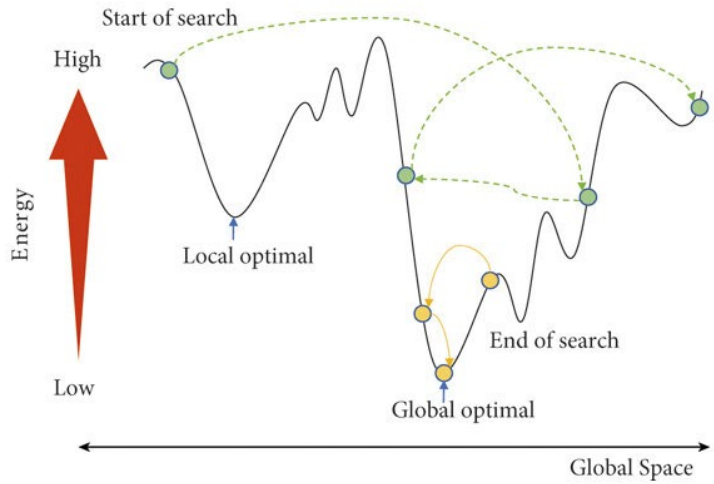
- **Exploitation:** Focus on regions that already perform well. Uses information from the current best solutions to generate new candidates. Example: gradient descent moves in the direction of the steepest descent.
- **Exploration:** Try new regions of parameter space. Introduces randomness or diversity to escape local optima. Example: mutation in GA.

GA favors exploration; BO explicitly models uncertainty to balance both.

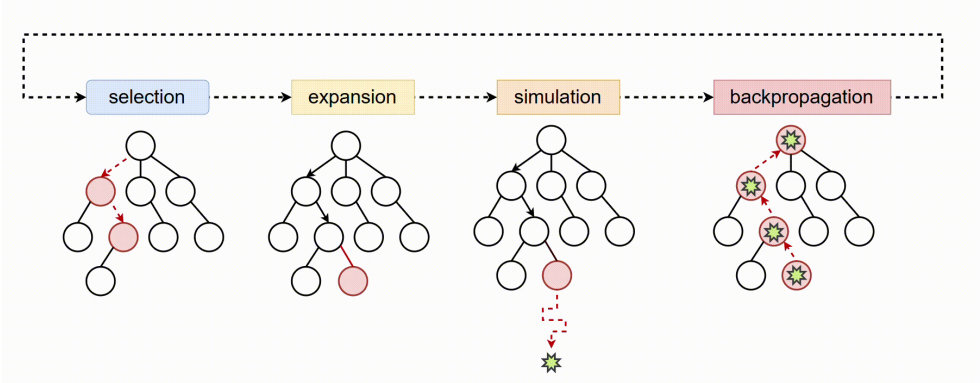
This trade-off determines whether optimization finds a **global** or **local** optimum.

Major Stochastic Prescriptive Algorithms

- **Genetic Algorithm (GA)**
- **Bayesian Optimization (BO)**
- **Simulated Annealing**
  - Inspired by metallurgy cooling process.
  - Accepts worse solutions early on to escape local minima.
- **Monte Carlo Tree Search**
  - Inspired by probabilistic sampling / casino game simulations.
  - Uses random simulations to explore future states and balances exploration with exploitation.



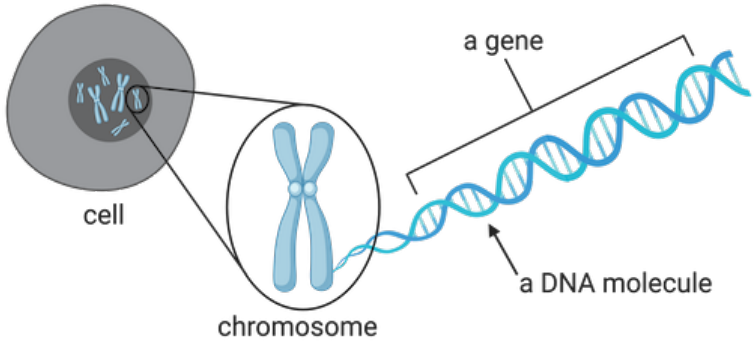
Simulated annealing algorithm
(DOI: 10.1155/2022/7363279)



Monte Carlo Tree Search
(source: Medium)

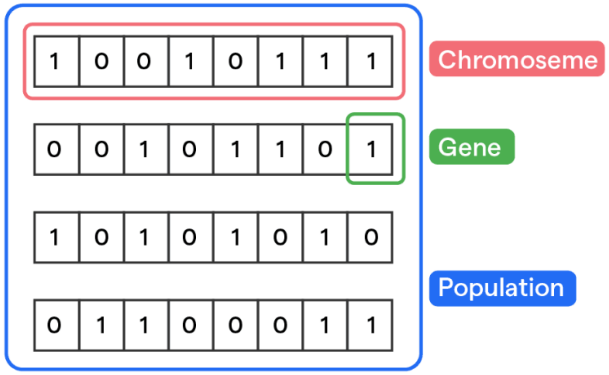
Genetic Algorithm (GA): Overview

Chromosomes and genes



Credit: Khan Academy

Genetic algorithm



Genetic Algorithms are inspired by the process of natural evolution, where the fittest individuals survive and pass on their traits.

- A GA starts with a population of possible solutions encoded like chromosomes.
- Each solution's quality is evaluated by a fitness function [$f(\mathbf{x})$], and new candidate solutions are generated through crossover (recombination) and mutation.
- Over successive generations, the population "evolves" toward better solutions.

Unlike gradient-based methods that require explicit knowledge of the function's form, GAs perform stochastic search guided by fitness and randomness, making them effective for complex, nonlinear, or discontinuous optimization problems common in chemical and materials engineering.

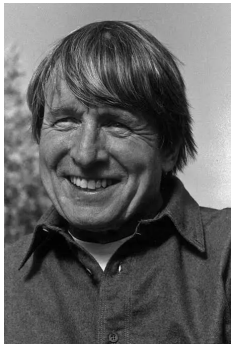
Genetic Algorithm (GA): History

1950s: Early computer simulations of evolution by Nils Aall Barricelli (1954) and Alexander Fraser (1957) established the first digital models of artificial selection and adaptation.

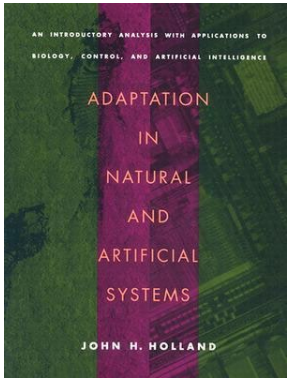
1960s–1970s: Ingo Rechenberg and Hans-Paul Schwefel developed evolutionary strategies for engineering optimization.

1975: John Holland published *Adaptation in Natural and Artificial Systems*, introducing theoretical foundations such as the Schema Theorem and popularizing the term "**genetic algorithm**".

1980s–1990s: Genetic algorithms gained widespread research and commercial use, marked by the first International Conference on Genetic Algorithms (1985) and commercial software like Evolver in 1989.



John Henry Holland

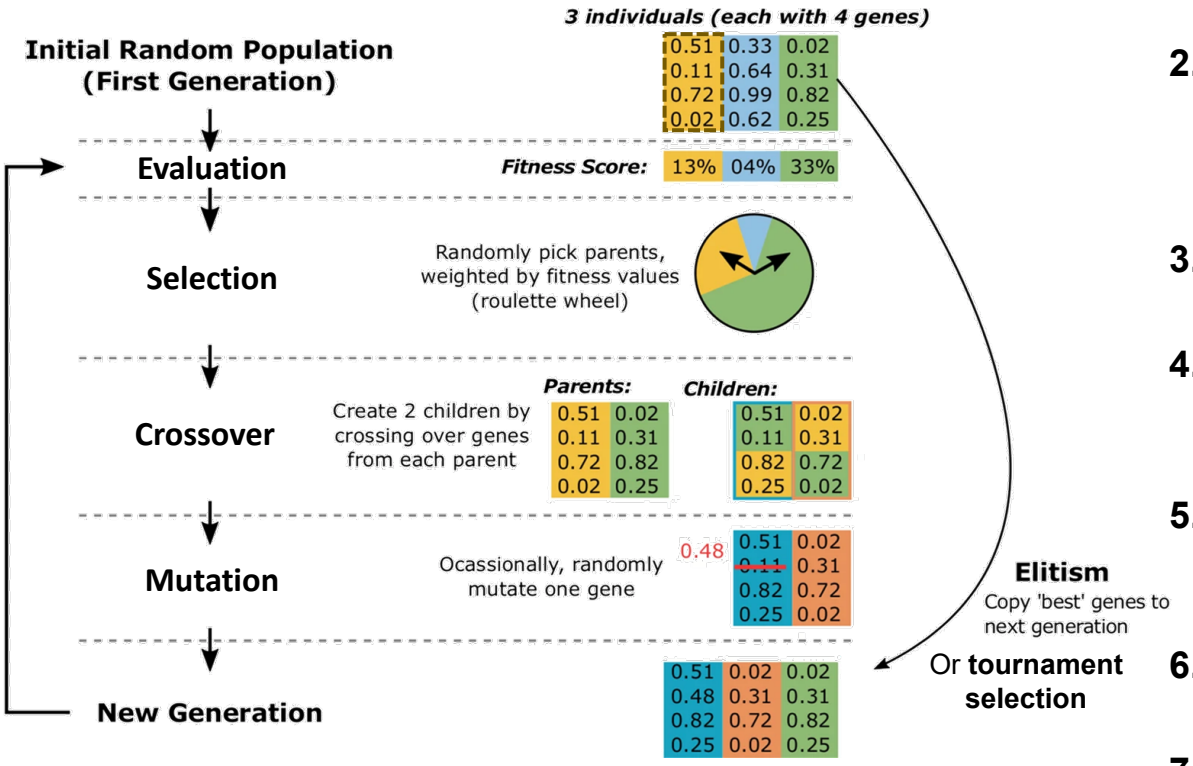


Adaptation in Natural and Artificial Systems by John Henry Holland. Being cited over 80k times as of Oct. 2025.

The 2006 NASA ST5 spacecraft antenna. This complicated shape was found by an evolutionary computer design program to create the best radiation pattern. It is known as an *evolved antenna*.

Image source: WWW

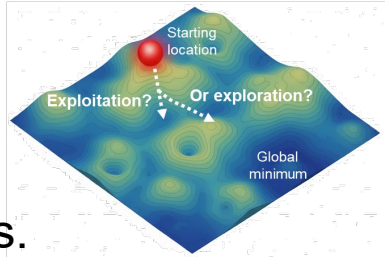
Genetic Algorithm (GA): Step-by-Step



1. **Initialization:** Randomly generate an initial population of N individuals.
2. **Evaluation:** Compute each individual's fitness/objective value $f(\mathbf{x}_i)$ via experiment or modeling.
3. **Selection:** Choose the fittest individuals for reproduction.
4. **Crossover:** Combine two parents to produce new offspring.
5. **Mutation:** Randomly alter offspring genes to maintain diversity.
6. **New generation:** Form the next generation and repeat.
7. **Best solution:** repeat Steps 2-6 until finding a "chromosome" that either minimize or maximize the fitness function.

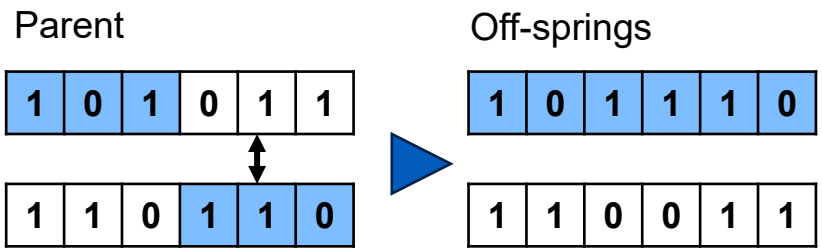
Woodward, R., Kelleher, E. Towards 'smart lasers': self-optimisation of an ultrafast pulse source using a genetic algorithm. Sci Rep 6, 37616 (2016).

Exploration and Exploitation in GA

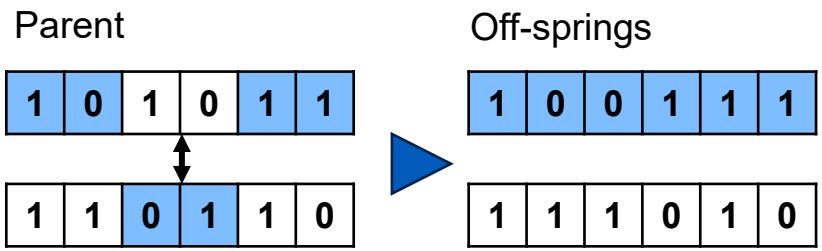


Crossover combines two parents to produce new offspring.

- **Single-point crossover:** Swap parts after a random cut point.
- **Two-point crossover:** Swap segments between two cut points.
- **Uniform crossover:** Each gene has a probability of being swapped.
- **Linear crossover:** child = parent1 +random number $*$ (parent2 − parent1)
- **Key Hyperparameter:** Crossover rate; higher values favor exploration.
- **Crossover promotes <u>exploitation</u>** by recombining existing good traits to refine promising regions of the search space.



Single-point crossover



Two-point crossover

Exploration and Exploitation in GA (cont'd)

Mutation randomly alters offspring genes to maintain diversity.

- **Gaussian mutation (continuous variables):** Add random noise drawn from a normal distribution, e.g., $x' = x + N(0, \sigma^2)$, where σ controls the mutation magnitude.
- **Uniform mutation (continuous variables):** Replace a gene with a random value within its allowed range.
- **Key Hyperparameter:** Mutation rate; higher values encourage exploration.
- **Mutation promotes <u>exploration</u>** by introducing new genetic material and preventing premature convergence.

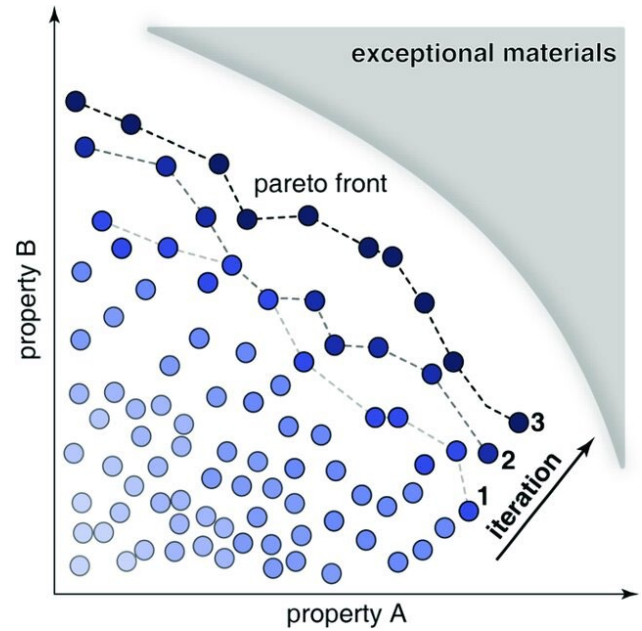
Mutation

Key Strengths and Limitations of GA

Strengths	Limitations
No derivatives required; works with black-box models	Computationally expensive (many evaluations)
Handles mixed/discrete variables	May converge slowly or prematurely
Finds global optima in complex landscapes	Requires parameter tuning (population size, mutation rate)
Easily parallelizable	Stochastic → results vary per run

Multi-Objective Optimization

- Many engineering problems require optimizing multiple conflicting objectives (e.g., maximize yield while minimizing energy or cost, selectivity-permeability tradeoff in membranes).
- A single “best” solution often does not exist. Instead, there is a set of trade-off solutions known as the Pareto front.
- In multi-objective GA, each solution is evaluated on several goals (e.g., maximize yield, minimize cost), and selection is based on Pareto dominance that keeps solutions that perform well overall while maintaining diversity along the trade-off front instead of a single “best” answer.
- Prescriptive AI can efficiently discover Pareto-optimal sets, allowing chemical engineers to visualize trade-offs and make informed decisions based on constraints and priorities.

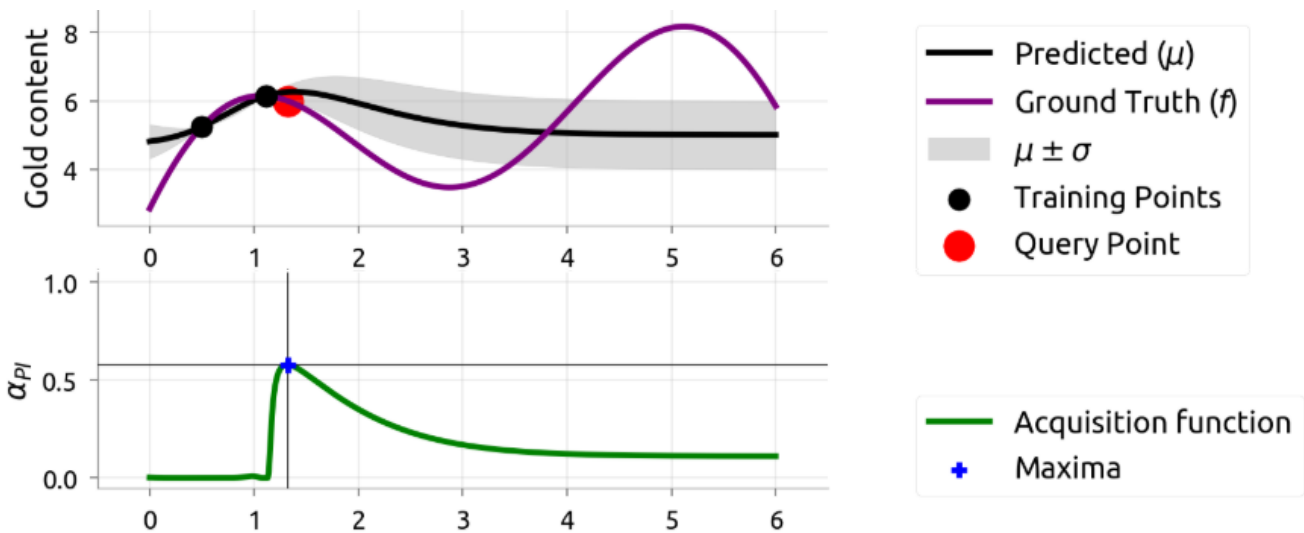


Example of pareto front, taken from 10.26434/chemrxiv-2023-x23s9

Bayesian Optimization: Overview

- **Bayesian Optimization (BO)** is a probabilistic, model-based optimization method for expensive black-box functions (e.g., experiments or simulations).
- BO builds a surrogate model (usually a Gaussian Process, GP) that predicts both mean (expected outcome) and uncertainty.
- BO balances **exploration** of uncertain regions and **exploitation** of promising regions via an acquisition function

Illustration of BO



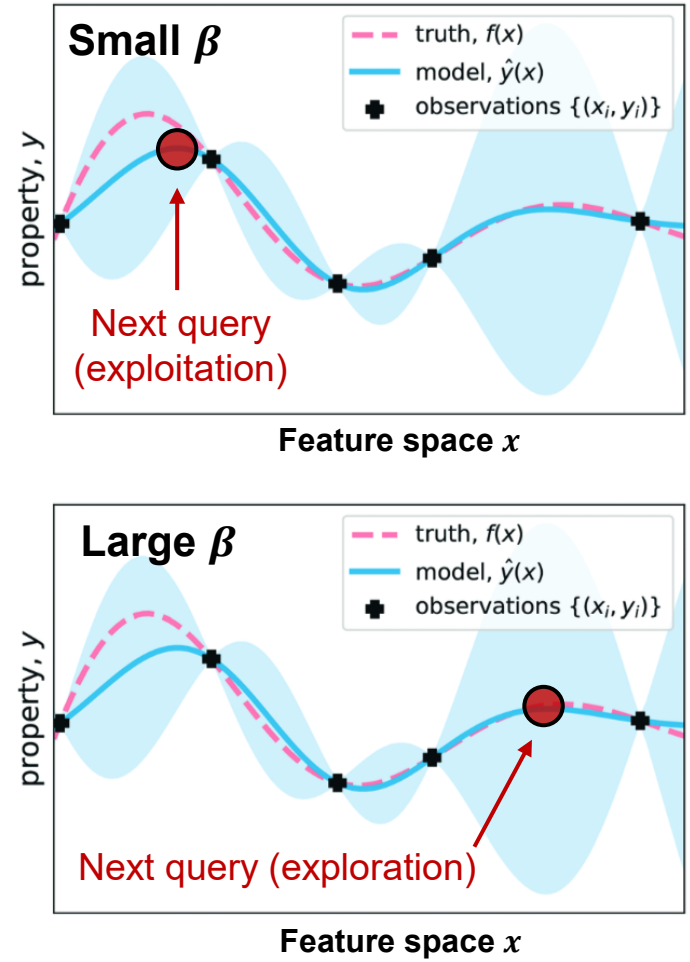
https://distill.pub/2020/bayesian-optimization/

Bayesian Optimization: Algorithm

- Start with a few initial evaluations of the function
- Fit a Gaussian Process surrogate (or other ML models with the ensemble technique) to obtain mean and uncertainty estimates
- Define an **acquisition function** that balances exploration and exploitation. E.g., upper confidence bound (UCB):

$$A(\mathbf{x}) = \hat{f}(\mathbf{x}) + \beta\sigma(\mathbf{x})$$

  where $\hat{f}(\mathbf{x})$ and $\sigma(\mathbf{x})$ are the predicted property of at input $\mathbf{x}$ and its associated uncertainty. The parameter β explicitly trades exploration and exploitation: if β is large (or small), the UCB is exploratory (or exploitative).
- Select the next evaluation by maximizing the acquisition function and update the model



Check this interactive demo of BO!
(made with Claude Artifacts)

https://distill.pub/2020/bayesian-optimization/
Mol. Syst. Des. Eng., 2021,6, 1066

Bayesian Optimization: Applications

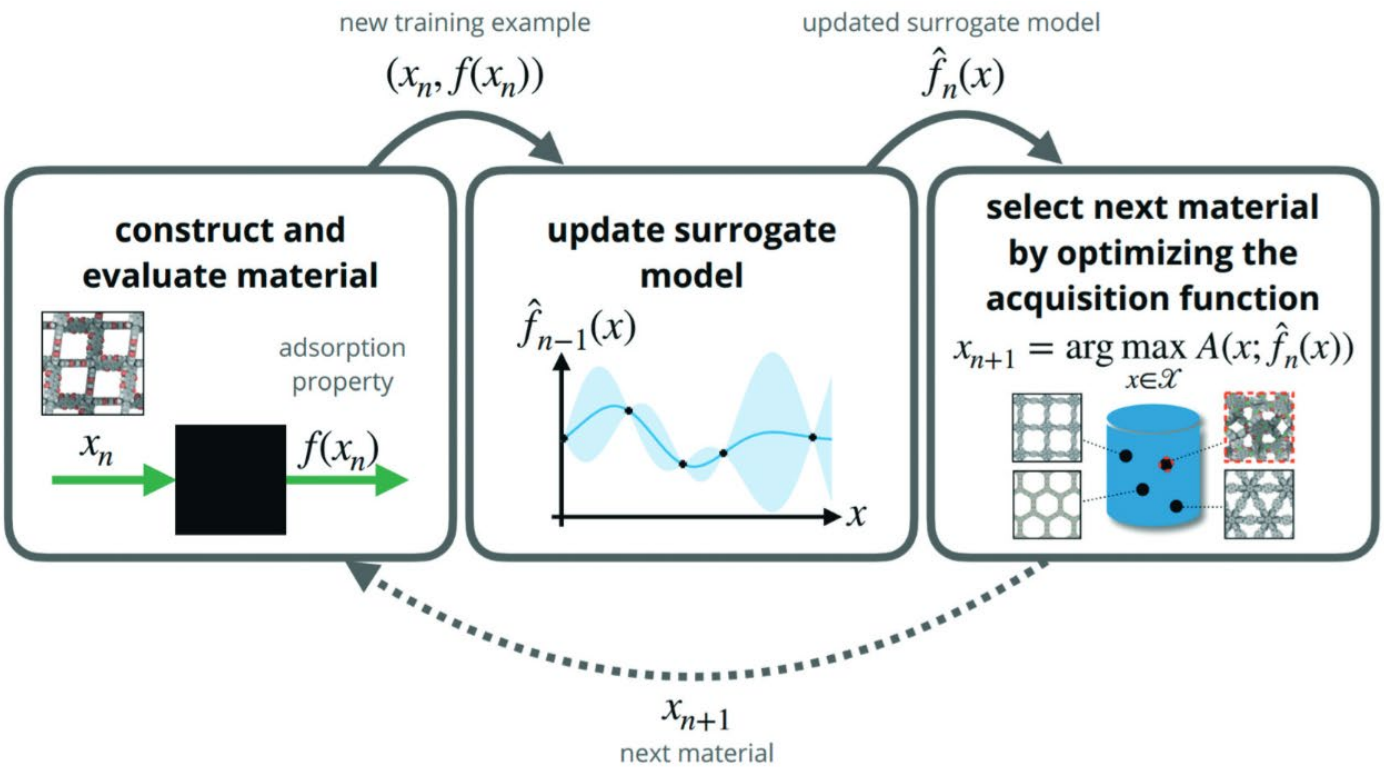


Fig. 2 Bayesian optimization (BO) is an active search method to find the input $\boldsymbol{x}$ that optimizes a black-box objective function $f(\boldsymbol{x})$. In the BO of nanoporous materials, we iterate within a feedback loop: (1) conduct an experiment that measures the property $f(\boldsymbol{x}_n)$ of the material represented by $\boldsymbol{x}_n$. (2) Using the new observation, update our belief about the underlying objective function $f(\boldsymbol{x})$, encapsulated by the surrogate model $\hat{f}(\boldsymbol{x})$. (3) Use the acquisition function $A(\boldsymbol{x})$ to pick the next material $\boldsymbol{x}_{n+1}$ for an experiment.

Mol. Syst. Des. Eng., 2021,6, 1066

Comparison between GA and BO

	Genetic Algorithm (GA)	Bayesian Optimization (BO)
Core Principle	**Evolutionary-inspired**. Uses a population of solutions that "evolve" via selection, crossover, and mutation.	**Statistical (model-based)**. Builds a probabilistic surrogate model of the function to intelligently select the next point to try.
Sample Efficiency	**Low**. It is a global search heuristic that explores broadly, which requires many samples.	**Very High**. This is its primary strength; it uses its model to avoid re-sampling unpromising areas.
Scalability (Dimensions)	**Good**. Can handle hundreds or even thousands of variables (dimensions).	**Poor**. Performance degrades significantly as dimensions increase. Often best for problems with < 20-30 dimensions.
Search Space Type	**Highly Flexible**. Natively handles continuous, discrete, integer, and mixed variable types.	**Primarily Continuous**. Requires special, often less effective, techniques to handle discrete or categorical variables.
Parallelism	**Easily Parallelizable**. The entire population of solutions can be evaluated simultaneously.	**Natively Sequential**. It must wait for the result of the current evaluation to update its model and decide the next point.

Practical Considerations and Best Practices

- Always define **constraints** and **bounds** to avoid unphysical solutions.
- Combine physical knowledge + AI (hybrid prescriptive systems).
- Normalize and scale design variables.
- Use **parallel evaluation** for computationally heavy tasks.
- Visualize optimization progress and convergence.